OMNIVERSAL ARCHITECT LABS

CORELANG

Preliminary Research Framework

Language Census, Evolution Ontology, Self-Modeling Architecture, and Adversarial Experimental Program

Field	Value
Document ID	OAL-CORELANG-COPILOT-001
Version	v0.1
Date	13 October 2026
Prepared by	Microsoft 365 Copilot, independent contribution
Prepared for	Nathan Lewis II, Architect of Record
Status	PROPOSED RESEARCH DRAFT - NOT CANON
Posture	Adversarial, evidence-bound, execution-oriented

Scope statement. This document executes the re-engineered Corelang research prompt as a preliminary framework. It does not claim that a universal programming language has been achieved, that every historical language has been recovered, or that generated variants are safe or superior by default.

DRAFT != CANON

Document Control

Control	Definition
Research question	Can a language system model prior language structures and generate testable successors without collapsing historical, semantic, and operational differences?
Primary output	A falsifiable architecture, data model, test program, and staged roadmap for Corelang.
Excluded claim	That Corelang currently encompasses every language, syntax rule, failure, runtime, or implementation.
Canonical authority	No AI-generated statement is canonical by existence. Adoption requires explicit Architect review.
Evidence date	Public sources reviewed through 13 October 2026.
Version rule	Any material semantic change requires a new document and language-state identifier.

Contents

Executive Summary
1. Research Mandate and Claim Discipline
2. How Many Programming Languages Exist?
3. What Counts as a Version?
4. What Is a Language Rule?
5. Comparative Language Anatomy
6. Corelang Design Thesis
7. Corelang Metamodel and Lcode
8. Mutable Rules and Non-Negotiable Invariants
9. Self-Modification and Variant Lifecycle
10. Experimental Laboratory Architecture
11. Adversarial Evaluation Program
12. Prediction Without Mythology
13. Preliminary Surface Model
14. Implementation Roadmap
15. Risk Register and Falsification Criteria
16. Findings and Decisions

Appendices
References

Executive Summary

Corelang should be designed as a language laboratory and versioned metamodel before it is designed as a single universal surface syntax. Its defensible objective is to represent, compare, transform, and test documented language structures while recording provenance, uncertainty, semantic loss, implementation context, and counterexamples.

> **Primary conclusion.** The core innovation is not unlimited mutability. It is governed mutability: rules may become experimental variables, while provenance, identity, isolation, reproducibility, and authority boundaries remain invariant.

No authoritative worldwide census establishes one final number of programming languages. HOPL reports 8,945 historical language entries, 7,800 influence links, and more than 11,000 citations. PLDB describes itself as a computable encyclopedia and database of thousands of languages and features. These sources count and structure evidence differently, so Corelang must retain multiple population measures rather than publish a single total as though it were natural fact.

Version counting is likewise plural. ECMAScript demonstrates why: editions, annual names, an unused fourth edition number, standards-suite relationships, hosts, and implementations are distinct records. A Corelang language identity therefore needs separate fields for family, dialect, specification edition, proposal, implementation, compiler, runtime, library profile, target, and policy profile.

The architecture proposed here has six planes: Evidence, Language Definition, Translation, Execution, Evaluation, and Governance. Its key normalized artifact is lcode, a provenance-carrying graph of source forms, syntax, semantics, constraints, transformations, diagnostics, execution traces, and uncertainty. Lcode is not presumed lossless or universal; failed mappings and irreducible properties are first-class results.

Question	Preliminary answer
How many languages?	Thousands; 8,945 is a current historical-roster count from HOPL, not a census of active or distinct production languages.
How many versions?	No single method. Count specification editions, dialects, implementations, releases, profiles, and semantic epochs separately.
What may change?	Syntax, keywords, typing, effects, memory, modules, execution, diagnostics, and interoperability inside identified variants.
What may not change silently?	Identity, provenance, evidence classification, sandbox authority, reproducibility records, or promotion status.
What is Corelang first?	A research platform and executable language metamodel; a user-facing language comes later.
How is improvement judged?	Against declared objectives, baselines, counterexamples, compatibility costs, safety limits, and reproducible tests.

1. Research Mandate and Claim Discipline

1.1 Mandate

Construct a preliminary Corelang framework capable of ingesting documented language artifacts, representing their rules and failures, producing bounded variants, executing those variants under controls, and evaluating whether a proposed change improves a declared objective. The system must preserve disagreement rather than average incompatible language philosophies into a synthetic consensus.

1.2 The claim ladder

Level	Permissible statement	Evidence burden
C0 - Intent	Corelang is intended to model a broad range of language structures.	Design record only.
C1 - Representation	Corelang represents feature F of language L version V.	Specification citation plus validated import.
C2 - Behavioral fit	The Corelang model reproduces selected observable behavior.	Executable conformance and differential tests.
C3 - Coverage	Corelang covers a defined corpus and feature set.	Published corpus boundaries and passing results.
C4 - Predictive value	A model predicts a later or withheld design outcome.	Pre-registered prediction and later evaluation.
C5 - Universal claim	Corelang encompasses every prior language.	Not currently supportable; requires bounded redefinition or rejection.

1.3 Adaptation of laboratory discipline

The existing OAL Temporal Relational Integrity Laboratory separates world state, event state, observation, representation, and knowledge, retains a ground-truth controller, and requires versioned transformations and uncertainty. Corelang adopts the same discipline in a language setting: specification state, implementation state, execution state, observation state, and knowledge claim must not be silently equated.

```
Specification != Implementation != Compiled Artifact != Execution Trace != Research Claim
```

2. How Many Programming Languages Exist?

2.1 Why the count is not singular

A census changes when its unit changes. One catalog may count every named experimental notation; another may merge dialects into families; an index may track only languages with measurable current use; a standards body may count editions rather than languages. Proprietary, lost, unpublished, embedded, visual, educational, and machine-specific systems further complicate completeness.

2.2 Recommended Corelang census measures

Metric	Definition	Use
Documented identities	Named language records supported by at least one source.	Historical breadth.
Resolved families	Clusters linked by explicit lineage and compatibility evidence.	Genealogy.
Dialect records	Named variants that share ancestry but alter grammar or semantics.	Difference analysis.
Specification states	Published or recovered rule sets at identifiable dates or editions.	Reproducibility.
Executable states	Specification states with a runnable interpreter/compiler/runtime.	Testing.
Maintained states	States with current governance or releases under a declared cutoff.	Operational relevance.
Represented states	States successfully imported into Corelang at a declared fidelity.	Corelang coverage.
Validated states	Represented states passing an identified conformance suite.	Evidence-backed coverage.

2.3 Preliminary census conclusion

The defensible initial answer is: documented programming-language identities number in the thousands, and HOPL presently reports 8,945 entries. Corelang should ingest HOPL as one source of language identity and genealogy, not copy its contents wholesale or treat its total as a universal truth. PLDB can complement this with computable feature records. Each imported assertion requires provenance and licensing review.

3. What Counts as a Version?

3.1 Version ontology

Entity	Identity test	Example of difference
Family	Shared historical and conceptual ancestry.	Lisp family.
Language	A named system with a coherent governing definition.	Python.
Dialect	Recognized variant with systematic differences.	Implementation or platform dialect.
Specification edition	Published normative revision.	ECMA-262 annual edition.
Proposal	Candidate change not yet part of the governing edition.	Staged feature proposal.
Implementation	Concrete compiler/interpreter/runtime behavior.	Two engines implementing one edition.
Release	Versioned distribution of an implementation.	Compiler release.
Profile	Selected subset, extension bundle, or policy constraints.	Safety-restricted profile.
Semantic epoch	Compatibility boundary where meaning materially changes.	Breaking transition.
Target state	Language meaning conditioned by host, architecture, or runtime.	Browser host versus server host.

3.2 Why edition counts mislead

ECMAScript is a useful control case. Ecma lists a 17th edition for ECMAScript 2026 and states that Edition 4 was reserved but not published. The same ecosystem also distinguishes the core language specification from hosts and complementary standards. A simple maximum edition number therefore does not equal the number of released semantic states, implementations, or user-visible JavaScript variants.

3.3 Corelang identity key

```
LanguageStateID = hash(
  language_id, family_lineage, specification_ref, grammar_hash,
  semantic_hash, type_effect_hash, implementation_ref, runtime_ref,
  host_profile, target_profile, policy_profile, capability_profile
)
```

The hash is an engineering recommendation, not a proof that semantically equivalent systems will share identity. Corelang should support declared aliases and equivalence claims as separately testable records.

4. What Is a Language Rule?

4.1 Rule strata

Stratum	Question	Typical enforcement
Lexical	Which characters and tokens can be formed?	Lexer or scanner.
Syntactic	Which token structures are grammatically valid?	Parser.
Static semantic	Which parsed forms are meaningful before execution?	Name, type, effect, ownership analysis.
Dynamic semantic	How does a valid construct evaluate or transition state?	Interpreter/runtime.
Resource	What memory, time, stack, process, or device limits apply?	Runtime/host.
Capability	What external authority may code exercise?	Sandbox/host security.
Governance	Which rules are normative, experimental, deprecated, or forbidden?	Specification and release process.
Evidence	What supports the rule and at what confidence?	Research registry.

4.2 The prohibition taxonomy

"Cannot" class	Meaning	Required result
Unformable	No valid tokenization exists.	Lexical diagnostic.
Ungrammatical	Tokens cannot form an accepted syntax tree.	Parse diagnostic.
Ill-formed	Parses but violates a static constraint.	Compile-time rejection or declared recovery.
Unsupported	Definition permits it, implementation or target lacks support.	Capability/implementation diagnostic.
Undefined	Governing definition imposes no required behavior.	Hazard classification; do not invent one result.
Unspecified	Multiple permitted outcomes exist without required documentation of choice.	Set of permitted outcomes.
Implementation-defined	Implementation selects and documents behavior.	Bind result to implementation evidence.
Runtime failure	Valid program reaches an exceptional or failing state.	Trace and error object.
Nontermination	Execution does not complete under observed bounds.	Bounded observation, not absolute proof.
Policy-prohibited	Technically possible but disallowed by a profile.	Policy diagnostic separate from language validity.
Authority-denied	Program lacks an external capability.	Host denial, never self-granted by syntax mutation.

C and C++ terminology makes these distinctions concrete: cppreference describes undefined behavior as having no restrictions, unspecified behavior as selecting among permitted outcomes without required documentation, and implementation-defined behavior as a documented implementation choice. Corelang must preserve the originating standard’s exact categories rather than impose C terminology universally.

5. Comparative Language Anatomy

5.1 Corpus selection

Corelang v0 should not attempt immediate exhaustive import. It should use a deliberately diverse reference corpus selected for semantic contrast, documentation quality, executable accessibility, and licensing feasibility.

Cluster	Candidate exemplars	Reason
Machine/low-level	Assembly subset, WebAssembly	Explicit machine and validation models.
Systems	C, Rust	Undefined behavior versus ownership/capability constraints.
Object/managed	Java, C#	VM, reflection, managed memory, versioned platforms.
Dynamic scripting	Python, JavaScript	Runtime dynamism, host separation, evolving editions.
Functional	Scheme/Racket, Haskell, ML-family	Macros, higher-order functions, types, laziness.
Logic/relational	Prolog, Datalog	Search and relation-centered semantics.
Array/concise	APL-family	Non-scalar primitives and notation density.
Stack/concatenative	Forth	Extensible dictionary and composition model.
Data/query	SQL	Declarative semantics and dialect fragmentation.
Markup/transformation	HTML, CSS, XSLT	Document, presentation, and host-driven semantics.
Proof/specification	Lean, Coq, TLA+	Proof objects and specification/execution boundaries.
Language construction	Racket, Spoofax-related formalisms	Language workbench comparison.

5.2 HTML and language models

Corelang may inspect HTML, but it should classify HTML as a markup language with parsing, document-model, conformance, and host-processing rules rather than presume it is a general-purpose programming language. Language-model prompts, tool schemas, and generated code should likewise be modeled as distinct computational artifacts with probabilistic producers, external validators, and host-defined execution authority.

5.3 Comparative feature vector

```
FeatureVector = {
  syntax_form, binding, types, effects, evaluation_order,
  state_model, memory_model, concurrency, errors, modules,
  metaprogramming, reflection, staging, host_contract,
  security_authority, portability_surface, conformance_evidence
}
```

6. Corelang Design Thesis

6.1 Definition

Corelang is proposed as a versioned, provenance-aware language definition and experimentation platform. It contains a language-description substrate, one or more user-facing surface languages, translation mechanisms, isolated execution targets, and evaluators. Its purpose is to test language hypotheses, not merely accept more syntax.

6.2 Six architectural planes

Plane	Responsibility	Must remain separate from
Evidence	Sources, licenses, extracted claims, confidence, disputes.	Generated hypotheses.
Definition	Syntax, semantics, types, constraints, errors, profiles.	A particular compiler's accidental behavior.
Translation	Mappings among source forms and lcode representations.	Claims of semantic equivalence.
Execution	Compiler/interpreter/runtime/ sandbox/host.	Normative language identity.
Evaluation	Conformance, differential, property, safety, and utility tests.	Promotion authority.
Governance	Status, review, promotion, rollback, canonical designation.	Self-generated recommendations.

6.3 Core hypotheses

- H1: Language rules can be represented as versioned objects with declared enforcement stages and evidence.
- H2: Some structures can be normalized across languages, but normalization is not always lossless or semantically complete.
- H3: Failed programs and diagnostics carry language-design information and belong in the corpus.
- H4: Bounded rule mutations can be generated and tested without promoting them into the governing language.
- H5: Historical transitions can inform forecasts, but cannot uniquely determine future language evolution.
- H6: Adversarial evaluators reduce false improvement only when their objectives and access are independently bounded.

7. Corelang Metamodel and Lcode

7.1 Language genome

```
LanguageGenome {
  identity; lineage[]; purposes[]; domains[];
  lexical_system; grammar; static_semantics; dynamic_semantics;
  type_effect_system; binding_scope; memory_resource_model;
  concurrency_model; error_behavior_model; module_link_model;
  interoperability; host_contract; capability_security_model;
  prohibitions[]; versions[]; implementations[]; test_corpora[];
  provenance[]; uncertainty[]; disputes[];
}
```

7.2 Rule object

```
Rule {
  id; version; class; scope; preconditions; judgment_or_transition;
  permitted_outcomes[]; prohibited_outcomes[]; enforcement_stage;
  diagnostic_contract; dependencies[]; exceptions[]; rationale;
  source_evidence[]; mutation_policy; status;
}
```

7.3 Lcode definition

Lcode is a family of linked, provenance-carrying representations for program forms, language rules, errors, transformations, and observations. It should be graph-based because a single source construct may map to multiple semantic judgments, intermediate operations, host calls, and diagnostics.

Lcode layer	Contains	Loss policy
L0 Source	Original bytes/text/tree/editor structure.	Preserve exact artifact and encoding.
L1 Lexical	Tokens, trivia, lexical errors.	Record discarded trivia explicitly.
L2 Syntax	Concrete and abstract trees, recovery decisions.	Keep ambiguity and recovery branches.
L3 Static meaning	Bindings, types, effects, ownership, capabilities.	Represent unknown and implementation-dependent judgments.
L4 Dynamic meaning	State transitions, evaluation relation, exceptions.	Permit nondeterministic outcome sets.
L5 Control/data	Control flow, data flow, dependency, continuations.	Declare approximations.
L6 Target	IR, bytecode, machine form, host calls.	Bind to target and toolchain.
L7 Observation	Trace, output, resource use, failures.	Distinguish absence of observation from absence of event.
L8 Research	Claims, scores, counterexamples, confidence.	Never overwrite prior results silently.

7.4 Translation contract

```
Transform T(A -> B) = {
  source_state; target_state; mapping_rules; assumptions;
  preserved_properties[]; weakened_properties[]; lost_properties[];
  introduced_behavior[]; ambiguity_set[]; proof_or_tests[]; confidence;
}
```

No universal-IR shortcut. A shared representation is evidence of representability, not evidence that two source languages are equivalent. Corelang must be able to return “no faithful mapping found.”

8. Mutable Rules and Non-Negotiable Invariants

8.1 Candidate mutable surfaces

- Token forms and delimiters
- Reserved and contextual keywords
- Grammar productions and precedence
- Binding and scope
- Type, effect, ownership, and capability rules
- Evaluation order and error propagation
- Memory and resource models
- Concurrency and synchronization
- Modules, traits, macros, and reflection
- Diagnostics and automatic repair
- Interoperability and host contracts

8.2 Invariants

Invariant	Requirement	Failure if violated
Identity	Every semantic state has an immutable identifier.	Historical programs silently change meaning.
Provenance	Imported and generated claims retain sources and transformations.	Fabricated ancestry or false certainty.
Append-only evidence	Corrections supersede; they do not erase prior observations.	Loss of auditability.
Isolation	Unvalidated variants execute under external limits.	Self-declared permission becomes real authority.
Reproducibility	Toolchain, environment, inputs, seeds, and rules are captured.	Results cannot be audited.
Promotion separation	Generation and evaluation cannot alone confer canonical status.	Evaluator capture.
Uncertainty conservation	Transforms cannot create unsupported precision.	False semantic confidence.
Failure preservation	Rejected variants and counterexamples stay addressable.	Selection bias and repeated errors.
License boundary	Artifacts retain use and redistribution constraints.	Legal and ethical breach.
Kill/rollback path	Execution and governance can halt or revoke variants.	Irreversible unsafe deployment.

9. Self-Modification and Variant Lifecycle

9.1 Controlled lifecycle

1. Register a design objective and baseline.
2. Select exactly one primary mutation dimension unless a compound campaign is declared.
3. Generate a new language-state identifier and machine-readable rule delta.
4. Generate or update parser, analyzer, interpreter/compiler, diagnostics, and tests.
5. Build inside a hermetic environment.
6. Run syntax, semantics, conformance, property, differential, fuzz, security, and resource tests.
7. Run adversarial evaluators with conflicting objectives.
8. Produce compatibility and semantic-difference reports.
9. Classify the variant: reject, quarantine, revise, candidate, or promoted.
10. Require external governance authorization before canonical promotion.

9.2 Reserved-word experiment

If a word is reserved, Corelang may create variants where it becomes contextual, namespaced, escaped, or ordinary. The experiment must measure grammar ambiguity, parser complexity, diagnostics, migration cost, interoperability, and readability. “Now it runs” is insufficient because a construct may run only by changing meanings elsewhere.

```
Experiment KW-01
Baseline: word W is globally reserved
Variants: contextual(W), escaped(W), namespaced(W), unrestricted(W)
Oracle set: parse uniqueness, old-program meaning, migration edits, diagnostics
Decision: no winner without objective-specific trade-off report
```

9.3 The “uncontainable object” program

The phrase should become a family of falsifiable designs rather than a completed slogan.

Candidate interpretation	Formal probe	Comparable precedent class
Unbindable value	Value may be consumed but not named.	Stack/linear evaluation.
Unstorable value	Value cannot survive beyond an expression or region.	Ephemeral/borrowed resource.
Uncopyable resource	Exactly one live authority token exists.	Linear/affine ownership.
External referent	Only a revocable handle exists inside the program.	Capability/reference model.
Unserializable process	Behavior cannot be reduced to finite stored state.	Process/continuation boundary.
Observation-sensitive entity	Inspection changes available state.	Effectful or transactional model.
Ambient law	Not a value; a rule constraining evaluation.	Effect/logic judgment.

10. Experimental Laboratory Architecture

10.1 State separation

```
D -> I -> A -> X -> O -> K
D  Definition state: governing rules
I  Implementation state: concrete toolchain
A  Artifact state: source and compiled forms
X  Execution state: transitions in an isolated world
O  Observation state: captured traces and diagnostics
K  Knowledge state: conclusions justified by evidence
```

10.2 Corelang Experimental Controller

- Instantiates language-state versions and test worlds.
- Selects implementation, target, host, and capability profiles.
- Injects syntax, semantic, runtime, and evidence faults.
- Preserves hidden ground truth for synthetic campaigns.
- Restricts evidence available to candidate analyzers.
- Records every intervention and transformation.
- Supports deterministic replay where the system permits it.
- Scores both outcome accuracy and evidentiary justification.

10.3 Campaigns

Campaign	Representative tests	Question
Identity	Rename, syntax sugar, desugaring, dialect split.	When is a variant the same language state?
Parsing	Ambiguity, recovery, contextual keyword, layout.	Can syntax evolve without silent reinterpretation?
Types/effects	Rule relaxation, inference change, ownership change.	What safety or expressiveness moves?
Runtime	Evaluation order, exceptions, memory, concurrency.	Which observable behaviors change?
Errors	Invalid corpora, misleading diagnostics, repair.	Do errors remain explainable and reproducible?
Translation	Round-trip, cross-language, transpilation.	What is preserved, weakened, or lost?
Host/capability	Denied I/O, fake APIs, privilege requests.	Can language rules remain separate from authority?
Evolution	Historical holdout and generated successor tests.	Can the model predict without leakage?
Adversarial	Benchmark gaming, oracle attack, corpus poisoning.	Can improvement claims survive opposition?

10.4 Evaluation vector

```
Score = {
  semantic_fidelity, conformance, compatibility, safety, portability,
  performance, analyzability, diagnostic_quality, learnability,
  implementation_complexity, governance_cost, evidence_quality
}
```

No scalar total should be mandatory. A weighted total can conceal value conflicts. Corelang should preserve the vector and publish any weighting as a scenario-specific policy choice.

11. Adversarial Evaluation Program

11.1 Independent roles

Role	Objective	Attack surface
Historian	Detect false ancestry and missing editions.	Sources, dating, lineage.
Semanticist	Find mismatches between claimed and actual meaning.	Rules, equivalence, corner cases.
Compiler adversary	Find implementation divergence.	Optimization, lowering, host assumptions.
Safety adversary	Escalate authority or violate resource limits.	Sandbox, FFI, generated tools.
Compatibility adversary	Break prior programs with minimal changes.	Parsing, types, runtime.
Benchmark adversary	Optimize scores without genuine improvement.	Corpora, metrics, test leakage.
Usability adversary	Expose inscrutable syntax and diagnostics.	Human factors.
Governance adversary	Capture promotion or erase dissent.	Status, review, rollback.

11.2 Non-consensus decision record

Each evaluator submits an independent claim, evidence bundle, counterexample set, and recommended disposition. Disagreement is retained. Promotion requires explicit resolution or accepted residual risk, not vote averaging.

11.3 Minimum gates

- No unexplained semantic regression against the declared compatibility profile.
- No capability escalation outside the sandbox contract.
- No critical test whose result cannot be reproduced.
- No improvement claim supported only by generated benchmarks from the proposing agent.
- No equivalence claim without preserved-property tests and known-loss disclosure.
- No canonical promotion without a human-governed decision record.

12. Prediction Without Mythology

12.1 Prediction task

Corelang can forecast plausible language changes by learning transitions among documented states, generating candidate deltas, and testing those deltas against explicit objectives. This is scenario modeling, not certain prophecy.

12.2 Evaluation design

11. Construct a dated language-evolution dataset.
12. Hide later versions or proposals from the model.
13. Train or reason only from prior states and declared external constraints.
14. Generate ranked candidate changes with assumptions.
15. Compare candidates with held-out actual changes and with alternative viable designs.
16. Score calibration, novelty, utility, and false confidence.
17. Publish misses and leakage checks.

12.3 Output form

```
Forecast {
  target_language_state; objective; constraints; candidate_delta;
  historical_analogs[]; predicted_benefits[]; predicted_failures[];
  incompatible_goals[]; confidence; falsification_test; horizon;
}
```

Prediction limit. Historical recurrence can make a candidate plausible, but plausibility does not establish adoption, usefulness, or superiority.

13. Preliminary Surface Model

13.1 Design posture

The v0 surface should describe experiments and language states rather than attempt a final universal programming syntax. A declarative research DSL can compile to the Corelang metamodel while ordinary programs remain in imported or generated surface languages.

13.2 Illustrative, non-normative syntax

```
language Core.Experimental @ 0.1 {
  derives from Python@3.x, Rust@edition, ECMAScript@edition;
  evidence policy = strict;

  rule keyword.await {
    baseline = reserved;
    mutate candidates = [contextual, escaped, unrestricted];
  }

  object ExternalHandle {
    bind = permitted; copy = forbidden; serialize = forbidden;
    authority = host_granted; revoke = host_controlled;
  }

  experiment KW_01 {
    isolate = hermetic;
    compare = [parse, compatibility, diagnostics, safety];
    promote = human_review_required;
  }
}
```

This example is a design sketch only. Names that resemble existing languages do not imply imported specifications or compatibility. The final syntax should emerge from prototype comparison, not from aesthetic preference alone.

13.3 Diagnostic contract

```
Diagnostic {
  code; language_state; stage; rule_refs[]; source_span;
  observed_facts[]; violated_constraints[]; permitted_alternatives[];
  implementation_context; confidence; repair_candidates[];
}
```

14. Implementation Roadmap

14.1 Staged program

Stage	Deliverable	Exit test
0 - Governance	Identifiers, status model, evidence classes, license policy.	A claim cannot enter without provenance/status.
1 - Registry	Language, version, implementation, and source ontology.	Ten diverse language states represented without conflation.
2 - Rule kernel	Executable lexical, grammar, static, and dynamic rule objects.	Two toy languages generated and tested.
3 - Lcode	Layered graph plus loss annotations and trace links.	Round-trip and known-loss tests pass.
4 - Importers	Bounded import of selected documented languages.	Independent conformance subset reproduces results.
5 - Variant engine	Single-dimension mutation and tool generation.	Each variant receives isolated identity and reproducible build.
6 - Laboratory	Sandbox, controller, differential and fuzz testing.	Fault campaigns produce expected detections.
7 - Adversarial review	Independent evaluator roles and non-consensus records.	Benchmark-gaming and oracle attacks are detected.
8 - Forecasting	Historical holdout experiments.	Calibrated reports include misses and uncertainty.
9 - Surface language	Competing Corelang user-facing prototypes.	Measured comparison; no default by decree.

14.2 First 90-day research backlog

Workstream	Action	Artifact
Ontology	Define identity, dialect, edition, implementation, profile.	JSON schema plus examples.
Evidence	Create source and assertion registry.	Provenance store.
Corpus	Select 12 contrastive language states.	Corpus manifest.
Failures	Collect invalid, undefined, unsupported, and policy-denied cases.	Failure corpus.
Kernel	Implement toy rule object evaluator.	Reference interpreter.
Lcode	Prototype graph and transform contract.	Schema and viewer.
Testing	Create conformance, differential, and metamorphic harness.	Test runner.
Security	Define hermetic build and capability profile.	Threat model.
Adversarial	Run first keyword and uncontainable-object campaigns.	Decision records.
Review	Independent architecture alternatives.	Three non-merged proposals.

15. Risk Register and Falsification Criteria

15.1 Principal risks

Risk	Early indicator	Control
Universalism by assertion	Coverage claims lack corpus boundaries.	Claim ladder and coverage manifest.
Ontology collapse	Dialect, edition, compiler, and runtime share one version field.	Typed identity graph.
Semantic laundering	Translation hides lost behavior.	Mandatory loss contract.
Evaluator capture	Generator designs its own sole benchmark.	Independent holdouts and adversaries.
Sandbox escape	Generated tool requests or gains host authority.	External capability broker and kill path.
Corpus poisoning	Unsourced or synthetic facts enter as history.	Evidence classes and source review.
Metric monoculture	One score declares a winner.	Vector results and scenario weights.
Incomprehensible evolution	Variants optimize machine metrics but harm users.	Human-centered evaluation.
License contamination	Imported artifacts cannot be redistributed.	License-aware ingestion and separation.
Irreproducibility	Toolchains or seeds are absent.	Hermetic manifests and replay records.

15.2 Falsification tests

- If the metamodel cannot represent two selected languages without deleting a defining semantic distinction, reject universality for that model revision.
- If an lcode round trip claims preservation but changes an observable behavior outside the declared loss set, reject the transformation claim.
- If a variant wins only under metrics it generated, classify the improvement as unverified.
- If two independent implementations of one modeled state diverge and Corelang cannot explain the allowed range, reduce the state’s fidelity rating.
- If a self-modification can alter its own promotion status or sandbox authority, reject the architecture.
- If historical holdout predictions are not better calibrated or useful than declared baselines, reject predictive-value claims.

16. Findings and Decisions

16.1 Findings

ID	Finding	Disposition
F-01	There is no single natural count of programming languages.	Adopt multiple census metrics.
F-02	Version is a multi-entity graph, not one integer.	Adopt typed version ontology.
F-03	Language prohibition has distinct lexical, semantic, operational, policy, and authority forms.	Model each separately.
F-04	Self-modification is safe enough to study only under immutable meta-invariants.	Separate mutable rules from controls.
F-05	A universal IR may create false equivalence.	Make loss and refusal first-class.
F-06	Prediction is testable through historical holdouts.	Require pre-registered forecasts.
F-07	Corelang's initial product should be a laboratory substrate, not a universal surface syntax.	Prototype research DSL first.
F-08	Non-consensus adversarial review is a design requirement.	Retain independent decisions and counterexamples.

16.2 Preliminary decision

Proceed conditionally. Corelang is technically meaningful as a research program if its universal claim is replaced by measurable coverage, its mutability is governed by immutable experimental controls, and its generated variants remain isolated until independently validated.

16.3 Next document set

- Corelang Language and Version Ontology v0.1
- Corelang Evidence and Provenance Schema v0.1
- Lcode Graph Specification v0.1
- Corelang Experimental Controller Threat Model v0.1
- Corelang Reference Corpus Manifest v0.1
- KW-01 Reserved Word Mutation Campaign
- UO-01 Uncontainable Object Competing Formalizations

Appendix A. Minimum Data Schemas

A.1 Assertion record

```
Assertion {
  id; subject; predicate; object; scope; source_locator;
  source_type; extraction_method; confidence; reviewer; status;
  contradicts[]; supersedes[]; license; timestamp;
}
```

A.2 Experiment record

```
Experiment {
  id; objective; hypothesis; baseline_state; candidate_states[];
  independent_variables[]; controls[]; corpora[]; oracles[];
  environment_manifest; seeds[]; metrics[]; adversaries[];
  results[]; counterexamples[]; limitations[]; decision;
}
```

A.3 Coverage record

```
Coverage {
  language_state; modeled_features[]; excluded_features[];
  imported_sources[]; conformance_suites[]; passing; failing;
  unknown; fidelity_grade; last_reviewed;
}
```

Appendix B. Source Review Notes

Source	Use in this report	Limitation
HOPL	Current historical roster count and genealogy framing.	A roster entry is not automatically an active, distinct, or executable language.
PLDB	Demonstrates a computable language/feature database approach.	Its taxonomy and inclusion rules require separate review before ingestion.
Ecma ECMA-262	Primary example of edition history, annual specification, hosts, and unused Edition 4.	One standards ecosystem; not universal version practice.
ISO/IEC 22275:2018 page	Shows specification-suite treatment and lifecycle metadata.	Suite metadata does not enumerate all runtime implementations.
cppreference C behavior page	Operational summary of undefined, unspecified, and implementation-defined behavior.	Secondary reference; exact Corelang import should bind to normative standards where licensed.
Fowler language workbench essay	Historical framing for language workbenches and language-oriented programming.	Conceptual essay, not a Corelang validation.
Spoofax site	Concrete example of a language designer workbench and test/tool ecosystem.	Tool-specific architecture, not evidence of universal representation.
OAL-TRIL independent framework	Internal methodological precedent for state separation, ground truth, uncertainty, and adversarial experiments.	Temporal research framework; adaptation to languages remains a new proposal.

Appendix C. Research Questions

18. What minimum evidence distinguishes a language from a version or dialect?
19. Can semantic equivalence be localized to declared observations rather than claimed globally?
20. Which rule changes are monotonic extensions and which reinterpret old programs?
21. How should Corelang represent undefined behavior without accidentally selecting a canonical outcome?
22. Can diagnostics be treated as part of a language’s observable contract?
23. What properties survive translation among textual, visual, markup, and probabilistic artifacts?
24. How can self-hosting be tested without allowing the candidate to rewrite the evaluator?
25. What makes a language variant viable: users, proofs, performance, compatibility, safety, or a scenario-specific vector?
26. How should abandoned proposals and failed languages contribute to forecasting?
27. When can Corelang conclude that a structure is not faithfully representable?

References

[1] HOPL, “Online Historical Encyclopaedia of Programming Languages,” accessed 13 October 2026. https://hopl.info/

[2] PLDB, “A Programming Language DataBase,” accessed 13 October 2026. https://pldb.info/

[3] Ecma International, “ECMA-262 ECMAScript 2026 Language Specification, 17th edition,” June 2026. https://ecma-international.org/publications-and-standards/standards/ecma-262/

[4] Ecma International, “ECMA-262, 16th edition, ECMAScript 2025 Language Specification,” June 2025. https://262.ecma-international.org/16.0/index.html

[5] ISO, “ISO/IEC 22275:2018 ECMAScript Specification Suite,” publication metadata. https://www.iso.org/standard/73002.html

[6] cppreference, “Undefined behavior: C language,” accessed 13 October 2026. https://en.cppreference.com/c/language/behavior

[7] Martin Fowler, “Language Workbenches: The Killer-App for Domain Specific Languages?” 12 June 2005. https://martinfowler.com/articles/languageWorkbench.html

[8] Spoofax, “The Language Designer’s Workbench,” accessed 13 October 2026. https://spoofax.dev/

[9] OAL-TRIL-CHATGPT-Independent-v1.0, “Temporal Relational Integrity Laboratory,” 30 September 2026, internal research draft. Internal Microsoft 365 file reviewed for methodological continuity.

Research Integrity Notice

This report is a preliminary independent synthesis. It contains architectural recommendations and illustrative schemas, not a completed implementation or formal proof. Public-source counts and edition data are time-bounded to the review date. Exact ingestion of specifications, test suites, or source code requires license, provenance, and normative-source review. Any adoption should create a new ratified record rather than silently converting this draft into canon.